

Melody Extraction from MIDI Files Using AI-ML Techniques

Navita

Center for Artificial Intelligence in
Medicine, Imaging & Forensics & Dept of
Physics
Sharda University
Greater Noida, India
navitamalik07@gmail.com

Ashok Kumar*

Center for Artificial Intelligence in
Medicine, Imaging & Forensics & Dept of
Physics
Sharda University
Greater Noida, India
ashok.kumar6@sharda.ac.in

Chanambam Mokaju Meitei

Center for Artificial Intelligence in
Medicine, Imaging & Forensics & Dept of
Physics
Sharda University
Greater Noida, India
mokajupri10@gmail.com

Abstract—Development of Artificial Intelligence and Machine Learning (AI-ML) based automated tools for melody extraction from audio music files are severely handicapped due to the scarcity of proper musical annotation to act as labels for supervised learning. For many kinds of traditional music the melodies can be quite intricate, and detailed annotation is not only scarce but also quite difficult to create. MIDI music files on the other hand carry within them the notes of melody which if extracted can act as musical annotations for training supervised melody extraction tools. In this study, several Machine Learning based classifiers were trained to determine whether a particular channel of MIDI file carries a melody or not. The six features used for training were Average TFIDF, Overlap, Number of Unique Notes, Note Duration, Note Velocity and Entropy. The testing accuracies obtained by Support Vector Machine (SVM), Random Forest, Logistic Regression, and Artificial Neural Network (ANN) classifiers were 86.67%, 88.64%, 86.91% and 93.80% respectively, with ANN based method giving the best testing result.

Keywords—Melody Extraction, Machine Learning, MIDI, Artificial Neural Networks

I. INTRODUCTION

Musical Instrument Digital Interface (MIDI) files are a crucial pillar in the ever-changing landscape of music creation and technology. MIDI has evolved from its modest beginnings as a standard for connecting electronic musical instruments in the early 1980s to become a universal language for the digital representation of musical ideas. MIDI files do not contain audio recordings; instead, they contain instructions that direct MIDI-compatible devices on how to make sound. This information includes musical notes, duration, pitch, velocity, and many control parameters. It is difficult for machines to recognize the patterns and structure of a specific style of music such as extraction of melody, rhythm, timbre, and chords from musical works.

Melody is the main theme of any form of music and is usually the most recognizable section [1]. Melody is generally thought of as a succession of pitches. Musical notes have pitch, duration, loudness and other characteristics. Chords consists of three notes performed simultaneously and are used to provide varying musical context on which melody is played.

Music synthesizers use a file format called Musical Instrument Digital Interface (MIDI) to exchange data between digital instruments. MIDI files contain complete instructions on how to generate music using electronic synthesizers. It is important to note that MIDI files do not contain any audio, but

they do contain information about which instruments to play, which notes to play, their duration, start and end times, and other details specified in the MIDI standard. To generate audio output, MIDI files can be played through a synthesizer and audio card. The MIDI 1.0 Detailed Specification was first published in 1985, and the updated MIDI 2.0 was introduced in 2020 [2].

In a monophonic melody, only one instrument or voice carries the melody. In MIDI files it corresponds to a single channel containing notes of the melody. In homophony, multiple voices or instruments can play the same melody moving together in harmony. If multiple channels play the melody either in unison or in parts, then for correct melody extraction it would be necessary to extract and combine melody pieces in different channels into one melody. Polyphony can involve different voices or instruments and different melodies simultaneously and possibly at different paces [3].

MIDI2vec [4] provides a new approach to enhance the representation of MIDI file data as vector embeddings similar to word2vec in Natural Language Processing, overcoming the limitations arising from the absence of high-quality metadata. This method relies on symbolic music data and enables machine learning applications, which can significantly improve automatic metadata tagging in music-related fields.

Long Short-Term Memory (LSTM) neural networks have been used in audio and musical studies for many purposes such as generating music pieces that mimic Bach's musical style [5]. The approach included several steps: converting MIDI files to song files, encoding them to input into the neural network, augmenting the files into different keys, training the neural network, and finally, generating music. The primary objective was to provide the neural network with an arbitrary note and gradually change it to produce a well-structured musical piece.

The idea of TF-IDF has previously been used in extracting a melody channel from MIDI files with the assumption that a channel of MIDI file containing musical notes is a “document” and each musical note in the channel is a “term” and the channel with minimum TF-IDF corresponds to the melody channel[11]–[17].

II. OBJECTIVES AND METHODOLOGY

The main objective of this study is to detect melody channels in a MIDI file using machine learning.

A. Data:

We downloaded 1963 MIDI files from the website <https://www.mididb.com/>. This website offers free MIDI song downloads for many genres of music. We downloaded songs from a variety of genres, including pop, classical hits, TV themes, country hits, rock songs, etc.

To establish the ground truth one of the authors listened to every channel of each MIDI file and determined the melody containing channels. The 1963 MIDI files had 15111 channel in total. Out of these, following two types of channels were dropped from consideration: 60 channels had some sort of melody played with chords

We dropped the channel containing melodies played with chords so that a fair comparison could be made with melody channels which contained succession of single notes without chords.

This left a total of 141981 channels out of which 10181 channels had no melody and were assigned ground truth label "0" implying the absence of melody and 4740 channels had monophonic melody and were assigned the ground truth label "1" implying the presence of melody.

B. Software Used

1) *PYTHON 3.9*: Python is an open-source, high-level, object-oriented programming language. We used the Anaconda distribution of Python

2) *LIBROSA (0.10.1)*: It is a powerful library of Python that is used for speech and musical audio signal processing [11].

3) *Pretty Midi(0.2.10)*: This is an important package for handling MIDI data [10]. Pretty midi provides several functions that help in extracting information from MIDI files or in generating new MIDI files.

4) *Tensorflow-Keras(2.15.0)*: Keras is a high-level library running on top of TensorFlow for machine learning with neural networks.

C. Midi File Related Concepts

MIDI 2.0 requires that all instruments or patches must map MIDI note number 69 to 440Hz pitch corresponding to the Piano note of middle A (A4), such that middle C (C4) on a piano corresponds to MIDI note number 60. The formula for converting between note number n and pitch frequency f in Hz is given as:

$$f = 440 \times 2^{(n-69)/12} \text{ Hz} \quad (1)$$

In addition, notes have a loudness or MIDI note-velocity value which ranges from 0 (quietest) to 127 (loudest). One MIDI cable that connects MIDI instruments carries data for all 16 channels. Data of all the 16 channels is supposed to constitute the MIDI track which can transmit all its data through a single MIDI cable. MIDI musical instruments can sometimes use multiple MIDI ports to transmit MIDI messages on more than 16 channels (16 channels per port). Each track can contain up to 16 channels and its MIDI messages can be transmitted using one independent MIDI port per track. In our case, we deal with single-track MIDI files using up to 16 channels in the track.

D. TF-IDF (Term Frequency-Inverse Document Frequency)

The Term Frequency-Inverse Document Frequency (TF-IDF) is a numerical measure that shows the importance of a word or term in a document in a chosen collection or corpus of documents. TF-IDF represents how important a word or term is in a document. A document contains text on a single theme, while a corpus of documents may contain different documents on different themes.

Our approach is different, and it does not depend upon whether a melody channel is already pre-identified in a MIDI file using some meta information since quite often melody channels are not explicitly identified in MIDI files. For each MIDI file we take our corpus to be all the channels in that MIDI file only, and we compute the TF-IDF of each note over this corpus of channels from one single MIDI file at a time. In our case the 'term' t corresponds to a MIDI note and 'document' d corresponds to a MIDI channel or track in the MIDI file.

1) *Term frequency (TF)*: The Term Frequency is defined as:

$$TF(t, d) = \frac{\text{Count of term } t \text{ in document } d}{\text{Total no. of terms in document } d} = \frac{f_d(t)}{\sum_{t'} f_d(t')} \quad (2)$$

where represents count of term t in document d . $TF(t, d)$ depends upon the term ' t ' and document ' d '. In our case ' t ' is a MIDI note and ' d ' is a MIDI channel

2) *Document Frequency (DF)*: $DF(t)$ corresponds to the number of documents in the corpus that contain the term t which in our case becomes the number of MIDI channels in a MIDI file containing the note n .

3) *Inverse Document Frequency (IDF)*: $IDF(t)$ corresponds to the size of the corpus divided by DF .

$$IDF(t) = \frac{N}{DF(t)} \quad (3)$$

To make the IDF values too large for rarer terms in large corpuses, a logarithm of the expression on the right can be taken. To avoid pathological cases when $DF = 0$ for a certain term, one can modify the denominator by adding one.

$$IDF(t) = \log \left(\frac{N}{DF(t)+1} \right) \quad (4)$$

But this can lead to negative values for terms that are present in all documents, i.e., if $DF = N$, then $IDF = \log(N/(N+1)) < 0$.

In our case we first find the set of all notes in a MIDI file, then find the set of distinct notes. This way each note is present in at least one channel, therefore $DF > 0$ always. The formula for TF-IDF then can be given as:

$$TFIDF(t, d) = TF(t, d) \times IDF(t) \quad (5)$$

4) *Counts Matrix and Occurrence Matrix*: We first generated a counts matrix M and an occurrence matrix O whose rows correspond to different MIDI note numbers and columns correspond to different channels or tracks. The count matrix M stores number of times a note appears in a

MIDI channel. The occurrence matrix O has 1 if a note is present in a channel and 0 if it is absent.

III. FEATURES FOR IDENTIFYING A MELODY CHANNEL

- A. *Average TFIDF* ($(TFIDF)$): The whole MIDI channel's average TF-IDF value $\overline{TFIDF}(c)$ can be defined as the average of the TF-IDF values of each note in the channel
- B. *Overlap*: A monophonic melody is a sequence of musical notes with no overlap between notes. For every MIDI file, we generated a CSV file for each channel, which included the beginning time, end time, and note number for each note played. The CSV file is sorted in the begin time column.
- C. *Number of Unique Notes*: Channels containing accompaniments do not use a large number of distinct notes [8]. On the other hand, channels containing melody are likely to have a larger number of unique notes. We count the total number of notes as per every instrument channel.
- D. *Note Duration*: Melodies do not contain notes with too short or too long durations. Notes less than 0.2 s or longer than 2s durations are produced by accompanying instruments [8].
- E. *Note Velocity*: This is related to loudness at which a note is played.
- F. *Entropy*: Entropy is a measure of the disorder or randomness of any system. We used entropy to filter the channels based on fluctuation because we know the melody channel will have more fluctuation. We calculated the entropy of notes in a channel of an instrument.

We had 141981 channels as part of our training dataset. Using the data from these 141981 channels selected from all the MIDI files, arrays of dimensions 141981 were created which were called feature vectors. The labels for each channel specifying whether the channel contains a melody or not were put in another 141981 dimensional array called the target vector. Target vector consisted of a column of labels taken as either 1 or 0 to identify whether the input is melody or a non-melody.

The Machine Learning models are trained on feature vectors to predict the target vector using supervised learning Machine Learning Models

IV. MACHINE LEARNING MODELS

A. Support Vector Machine Classifier (SVM)

We first used Machine Learning algorithm called Support Vector Machine to classify the MIDI files based on their features. The SVM aims to find a hyperplane that best separates data points into different classes.

Hyper parameters: The values of parameters used during model training using *Python Machine Learning package* Scikit-Learn's Support vector machine classifier (SVC) were obtained using `get_params()` method as follows: 'C'=1.0,'break_ties'=False,'cache_size'=200,'class_weight'=None,'coef0'=0.0,'decision_function_shape'='ovr','degree'=3,'gamma'='scale','kernel'='rbf','max_iter'=1,'probability'=False,'random_state'=0,'shrinking'=True,'tol'=0.001,'verbose'=False

TABLE I. SOME HYPERPARAMETERS AND THEIR VALUES FOR SUPPORT VECTOR MACHINE CLASSIFIER

Name of Parameter	Definition	Value
Regularization parameter.	In machine learning models, regularization is a technique used to stop overfitting. In SVM, the regularization parameter (C) regulates the trade-off between having a simple decision boundary and having a well-fitting training set. (value=1.0)	1.0
SVM Kernel function	Method of transforming input data into a higher-dimensional space (Radial Basis Function kernel or "rbf" was used)	'RBF'
Shrinking Heuristic	Works by removing support vectors that are likely not to affect the decision boundary. ("true")	'True'

B. Random Forest Classifier

Random Forest is an ensemble learning Machine Learning algorithm that combines multiple decision trees to make predictions. Each tree in the forest is trained on a random subset of the training data and makes its predictions. The final prediction is determined by combining the predictions of all the individual trees in the forest. This helps to improve the accuracy and robustness of the overall model, as it reduces the risk of overfitting the training data and increases the generalizability of the model to new and unseen data. The Random Forest algorithm is commonly used in machine learning applications such as classification, regression, and feature selection.

Hyper parameters: The values of parameters used during model training using ScikitLearn. Random Forest classifier were obtained using `get_params()` method as follows: `bootstrap=True, ccp_alpha=0.0, class_weight=None, criterion='entropy', max_depth=None, max_features='sqrt', max_leaf_nodes=None, max_samples=None, min_impurity_decrease=0.0, min_samples_leaf=1, min_samples_split=2, min_weight_fraction_leaf=0.0, n_estimators=10, n_jobs=None, ob_score=False, random_state=None, verbose=0, warm_start=False`.

TABLE II. SOME HYPERPARAMETERS AND THEIR VALUES FOR RANDOM FOREST CLASSIFIER

Name of Parameter	Definition	Value
Number of Estimators	The number of trees in the forest. Increasing the number of trees generally improves performance	10
Splitting Criterion	The function is used to measure the quality of a split,	'Entropy'
Minimum Samples for Split	The minimum number of samples required to split an internal node	2
Minimum Samples per Leaf	The minimum number of samples required to be at a leaf node. Similar to, but applied to leaf nodes)	1
Bootstrap Sampling	This is the constant that is multiplied by the product of the data point and the kernel value..	'True'

C. Logistic Regression

Logistic Regression is a type of machine learning algorithm that is used for classification tasks rather than regression tasks. This algorithm models the probability of an

instance belonging to a particular category. The output of the logistic regression model is transformed using the logistic function (also known as the sigmoid function) to produce a probability score between 0 and 1.

Hyper parameters The values of parameters used during model training using ScikitLearn. Logistic Regression were obtained using `get_params()` method as follows: `C'=1.0,class_weight=None,dual=False,fit_intercept=True,intercept_scaling=1,l1_ratio=None,max_iter=100,multi_class='auto',n_jobs=None,penalty='l2',random_state=0,solver='lbfgs',tol=0.0001,verbose=1,warm_start=False`.

TABLE III. SOME HYPERPARAMETERS AND THEIR VALUES FOR LOGISTIC REGRESSION

Name of Parameter	Definition	Value
Regularization Penalty	Specifies the standard that is applied while penalizing. 'l1' and 'l2' L1 and L2 are combined to form 'elasticnet'	'L2'
Inverse of Regularization Strength	It controls the trade-off between fitting the training data well and preventing overfitting.	1.0
Maximum Iterations	Maximum number of iterations for the solvers to converge	100
Multiclass Handling	Decides how multi-class situations should be handled	'Auto'

D. Artificial Neural Network Model

An Artificial Neural Network (ANN), is a computational model that is inspired by our brain's biological neural networks. It is extensively used in machine learning for tasks such as regression, classification, pattern recognition, and more. ANNs are made up of interconnected nodes, also known as perceptrons or artificial neurons, that are arranged in layers. There are three types of layers in an ANN: input layer, hidden layers, and output layer.

1) Architecture

- The model is built using Keras' `Sequential()` method and composed of a series of layers.
- The first layer is a Dense layer with 32 neurons, a ReLU activation function, uniform weight initialization, and an input shape.
- Two additional Dense layers with 32 units and ReLU activation functions follow.
- A dropout layer with dropout rate of 0.01 is added to reduce overfitting.
- Output layer has a single neuron with a sigmoid activation function, which is suitable for binary classification tasks, is used.

2) Model Compilation

- The model has been compiled using the Adam optimizer, with a learning rate set to 0.0001.
- Since our task is a binary classification problem, we have chosen binary cross-entropy as the loss function.
- The evaluation metrics are accuracy and binary cross entropy loss.

3) Model Summary & Training

- The output of `model.summary()` method of Keras is shown in Fig. 1, which shows the details of different layers and associated number of trainable parameters.

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 32)	224
dense_5 (Dense)	(None, 32)	1056
dense_6 (Dense)	(None, 8)	264
dropout_1 (Dropout)	(None, 8)	0
dense_7 (Dense)	(None, 1)	9

=====

Total params: 1553 (6.07 KB)
Trainable params: 1553 (6.07 KB)
Non-trainable params: 0 (0.00 Byte)

Fig. 1. Summary of Training Model

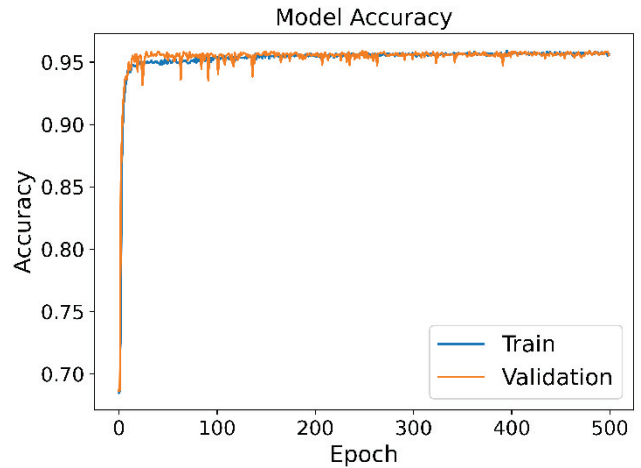


Fig. 2. Training and validation accuracy of the ANN model

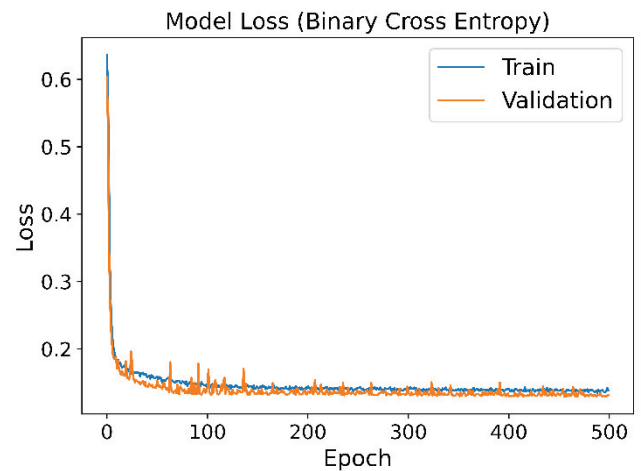


Fig. 3. Training & validation loss of the ANN model

E. Testing Result of the ANN Model

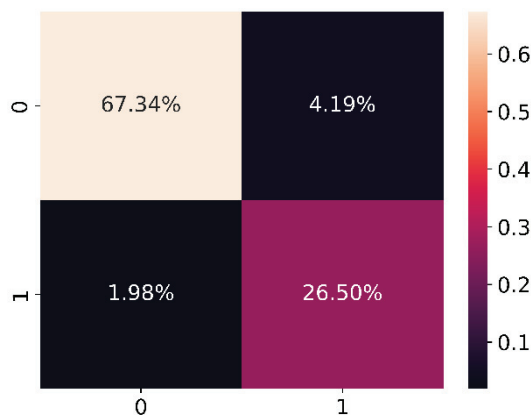


Fig. 4. Confusion matrix showing the testing accuracy of the ANN model

V. RESULTS & DISCUSSIONS

10% of the dataset was separated from the training dataset before training and was never seen by the AI-ML models during their training phase. Testing is done on these unseen data. Fig. 5 lists the classification accuracies achieved by different models. The SVM model had classification accuracy of 86.67%, the Random Forest model had 88.74%, Logistic Regression had 86.93% and the ANN model had the highest testing accuracy of 93.8%.

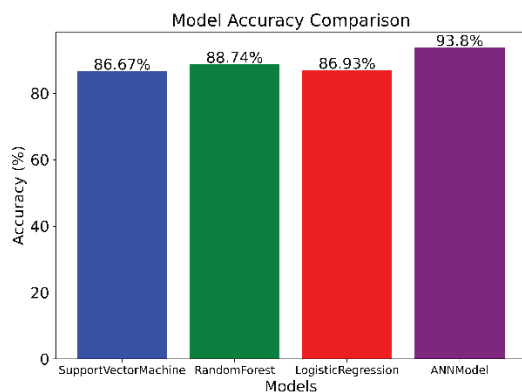


Fig. 5. Comparison of testing accuracies of different models

VI. CONCLUSION & FUTURE SCOPE

In this paper, we investigated the performance of AI-ML techniques in identifying the channel containing melody in MIDI files. To achieve this, we applied various ML algorithms such as Support Vector Machines, Random Forests, Logistic Regression, and Artificial Neural Networks.

Our results show that it is possible to achieve high accuracy in identifying the melody channel of MIDI files. The melodies extracted from MIDI channels identified using this method, could help in developing annotated music samples for supervised learning based training of AI models for automated melody extraction from audio samples. This adds support to the view that machine learning techniques can be an effective tool in music analysis. Automated melody extraction from audio samples is often hampered due to the scarcity of

annotations audio samples. Supervised AI models can be pre-trained on audio versions of MIDI files with melody annotations extracted from the melody channel acting as ground truth for supervised learning. Further, Deep Learning based approaches involving Recursive Neural Networks (RNNs), Long Short Term Memory (LSTMs) Networks or Transformers could be used fruitfully for melody extraction from audio samples,

REFERENCES

- [1] Z. Jiang and R. B. Dannenberg, "Melody Identification in Standard MIDI Files." [Online]. Available: <https://en.wikipedia.org/wiki/Viterbi>
- [2] P. D. Lehrman, "MIDI 2.0: Promises and Challenges," in *Music Encoding Conference Proceedings*, 2020, pp. 55–58.
- [3] G. Ozcan, C. Isikhan, and A. Alpkocak, "Melody extraction on MIDI music files," in *Proceedings - Seventh IEEE International Symposium on Multimedia, ISM 2005*, 2005, pp. 414–421. doi: 10.1109/ISM.2005.77.
- [4] P. Lisena, A. Meroño-Peñuela, and R. Troncy, "MIDI2vec: Learning MIDI embeddings for reliable prediction of symbolic music metadata," *Semant. Web*, vol. 13, no. 3, pp. 357–377, 2022, doi: 10.3233/SW-210446.
- [5] A. Fang, P. He, F. Pan, L. Yang, M. Deng, and W. Yu, "A Music Synthesizing Method for Tinnitus Sound Therapy Based on LSTM and Transformer," in *2022 7th International Conference on Intelligent Computing and Signal Processing, ICSP 2022*, Institute of Electrical and Electronics Engineers Inc., 2022, pp. 1044–1049. doi: 10.1109/ICSP54964.2022.9778588.
- [6] A. Uitdenbogerd and J. Zobel, "Music ranking techniques evaluated," in *Proceedings of the 25th Australasian Computer Science Conference*, Australian Computer Society, 2002, pp. 275–283.
- [7] M.-K. Shan and F.-F. Kuo, "Music style mining and classification by melody," *IEICE Trans. Inf. Syst.*, vol. 86, no. 3, pp. 655–659, 2003.
- [8] S. Velusamy, B. Thoshkahna, and K. R. Ramakrishnan, "A novel melody line identification algorithm for polyphonic MIDI music," in *International Conference on Multimedia Modeling*, Springer, 2007, pp. 248–257.
- [9] C. Isikhan and G. Ozcan, "A survey of melody extraction techniques for music information retrieval," in *Proceedings of 4th Conference on Interdisciplinary Musicology (SIM'08)*, Thessaloniki, Greece, 2008.
- [10] C. Raffel and D. P. W. Ellis, "INTUITIVE ANALYSIS, CREATION AND MANIPULATION OF MIDI DATA WITH pretty_midi." [Online]. Available: <http://craffel.github.io/pretty-midi/>
- [11] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manag.*, vol. 24, no. 5, pp. 513–523, 1988.
- [12] S. Li, S. Jang, and Y. Sung, "Melody extraction and encoding method for generating healthcare music automatically," *Electron.*, vol. 8, no. 11, Nov. 2019, doi: 10.3390/electronics8111250.
- [13] S. Qaiser and R. Ali, "Text mining: use of TF-IDF to examine the relevance of words to documents," *Int. J. Comput. Appl.*, vol. 181, no. 1, pp. 25–29, 2018.
- [14] J. Ramos, "Using tf-idf to determine word relevance in document queries," in *Proceedings of the first instructional conference on machine learning*, Citeseer, 2003, pp. 29–48.
- [15] A. Aizawa, "An information-theoretic perspective of tf-idf measures," *Inf. Process. Manag.*, vol. 39, no. 1, pp. 45–65, 2003.
- [16] L. Havrlant and V. Kreinovich, "A simple probabilistic explanation of term frequency-inverse document frequency (tf-idf) heuristic (and variations motivated by this explanation)," *Int. J. Gen. Syst.*, vol. 46, no. 1, pp. 27–36, 2017.
- [17] H. Christian, M. P. Agus, and D. Suhartono, "Single document automatic text summarization using term frequency-inverse document frequency (TF-IDF)," *ComTech Comput. Math. Eng. Appl.*, vol. 7, no. 4, pp. 285–294, 2016.